Parked, but Reclaimable: Measuring the Missing State in Agentic LLM Autoscaling

Anonymous Author(s)
Submission #xxx

Abstract

Agent programs alternate model invocations with tool calls. During a tool call, GPU utilisation and request queues can both be zero even though the program will soon reuse a long prefix. We initially assumed that native vLLM reserved that program's KV blocks and that KV-cache utilisation therefore exposed the parked state. A hardware validation falsified this premise. In vLLM 0.25.1, a completed request releases its blocks to a reclaimable prefix cache: an immediate reuse may be cheap, but the exported active-KV metric also reads zero and new work can evict the prefix.

We report the measurement, correct a trace-driven simulator, and re-evaluate seven autoscaling policies. On Qwen2.5-14B, a 32k-token cache miss adds 9.03 s relative to a hit, while all eight parked sessions produce zero GPU, running, waiting, and active-KV readings in 114 samples. A pressure experiment shows that a 16k prefix remains intact behind at most eight concurrent 4k contexts, is about 64.5% retained behind 16, and is gone behind 24; delays of 0–32 s have little independent effect. The same run calibrates batching ($k_{1/2} = 4.606$) and a 38.155 s process cold start.

The correction reverses the original policy result. Across six agentic points on the Azure coding trace, mean SLO attainment is 0.941 for KV-util, 0.929 for HPA, and 0.901 for a candidate policy that counts parked programs. Prefix pressure, not scale-in alone, dominates recomputation, and the relative order changes with cache capacity and batch size. The useful signal is therefore not "parked occupancy" but the uncertain future value and survival probability of reclaimable prefixes. We release the falsified version, raw measurements, corrected simulator, and negative result together.

Keywords

LLM serving, agents, autoscaling, prefix cache, negative results

1 Introduction

An LLM agent is a program rather than a request. It invokes a model, runs an external tool, appends the result, and invokes the model again on an extended transcript [2]. Between invocations it uses no GPU. We call this interval *parked*. The accumulated prefix can still be valuable: losing it forces a re-prefill before the next turn.

This creates a tempting but incorrect mental model. If a serving engine pins a parked program's KV blocks, compute-side signals miss the program while a memory-side signal sees it. Scaling in then destroys hard-held state. Our original study encoded exactly that model and produced a strong result for a controller that counted parked programs.

We designed a four-part vLLM validation to measure the assumptions before using them in the paper. The experiment did what validation should do: it falsified the central one. Native vLLM returns a completed request's blocks to its free/reclaimable pool. Prefix caching may preserve their content, but the program has no reservation and vLLM's exported active-KV metric reports no allocation for it. During a tool call all three deployed signals can therefore read zero simultaneously:

- GPU utilisation sees no kernel executing for the program;
- queue/running metrics see no submitted request; and
- active-KV utilisation sees no blocks allocated to an active request.

The state is neither resident in the resource-accounting sense nor absent in the performance sense. It is an opportunistic, evictable option on a future cache hit. This distinction changes both the model and the control problem.

This paper makes three contributions. First, it measures the missing state and its price on a real vLLM instance. Second, it corrects the simulator to separate active allocation from reclaimable prefix content, including token-level partial LRU eviction. Third, it reports that the original ParkAware mechanism no longer wins reliably. This is not a weaker confirmation of the old claim; it is a reproducible negative result that identifies what a future controller must estimate: prefix reuse value under cache pressure.

2 Workload and Signals

2.1 Agentic serving

Context accumulates across an agent trajectory. Turn i contains prior model outputs and tool results, so a later prompt may be tens of thousands of tokens. Serving systems use paged KV management and prefix caching to avoid repeating that prefill [3]. Published coding-agent runs contain long trajectories; we therefore sweep up to 64 turns rather than treating every invocation as an independent chat request [10].

Microsoft's public inference dataset provides one conversation trace and one coding trace [1]. The coding trace contains 8,819 requests with a context-to-generation ratio of 73:1, index of dispersion 71.3, and peak-to-mean arrival rate 12.8. The conversation trace contains 19,366 requests, with the corresponding values 5.5:1, 4.0, and 1.8 (Figure 1). The coding trace is not a trace of tool calls, but its long-context, short-output and bursty shape makes it a useful request-level substrate.

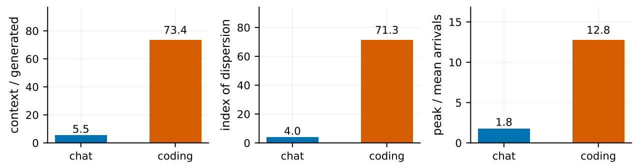

Figure 1: Measured properties of the two Azure traces.

2.2 Three zeroes, one latent value

GPU utilisation, queue depth, and active-KV utilisation are sensible signals for submitted requests. A parked program is outside that interface. Yet a cache hit can make its next turn much cheaper. We denote a program's cached prefix by c_i and the fraction still present when it returns by $q_i \in [0, 1]$. Its latent recompute liability is

$$L_i = \frac{(1 - q_i)c_i}{R_{\text{prefill}}}. \tag{1}$$

Neither q_i nor the program's probability of returning is exported by the three native autoscaling signals. Counting programs alone supplies neither.

3 Hardware Validation

3.1 Setup

We ran vLLM 0.25.1 with prefix caching enabled, using the Qwen2.5-14B Instruct checkpoint on one device reported as RTX 4090 with 49,140 MiB. The server reported capacity for 75,216 KV-cache tokens. Each point has three repetitions unless noted. The repository contains the exact scripts, raw CSVs, server logs, and a machine-readable summary.

3.2 V1: price of a prefix

We issue the same prefix twice for a hit and a distinct prefix for a miss. At 1k tokens, the mean miss–hit penalty is 0.161 s; at 32k it is 9.028 s and the hit is 44.2× faster. A linear fit gives 0.286 s per 1k tokens with $R^2 = 0.989$. The state is valuable even though it is not reserved.

3.3 V2: the assumption that failed

Eight sessions alternate model calls with 8 s tool sleeps while DCGM and the vLLM Prometheus endpoint are sampled. Across 114 samples in which all sessions are parked, mean GPU utilisation, running requests, waiting requests, and active-KV utilisation are all exactly zero. When any session computes, the corresponding means are 93.5% GPU, 5.96 running, 0.62 waiting, and 29.5% active KV. Thus KV-util does not rescue observability of parked state.

3.4 V3–V4: calibration

At concurrency $k \in \{1, 2, 4, 8, 16, 32\}$, per-sequence output rate fits

$$r(k) = \frac{R}{k + k_{1/2}} \tag{2}$$

with $R = 156.19$, $k_{1/2} = 4.606$, and $R^2 = 0.99995$. The original assumed $k_{1/2} = 16$, so its batching model was materially too optimistic. Three clean server process starts with a warm host page cache take 38.163, 38.141, and 38.161 s (mean 38.155 s). This is not a disk-cold or 70B cold start.

3.5 P1: survival under delay and pressure

We park a 16k-token target, wait $\tau \in \{0, 8, 32\}$ s, introduce $N \in \{0, 4, 8, 16, 24\}$ distinct 4k-token neighbors, and probe the target. We normalize probe TTFT between the same-trial hit and miss to estimate retained prefix fraction. All 45 trials completed. At $N \leq 8$, retention is essentially one; at $N = 16$, it is 0.645; and at $N = 24$, it is approximately zero. The three delay curves nearly overlap. In this range, allocation pressure rather than wall-clock age determines survival.

This result also exposes partial eviction. The measured capacity is 75,216 tokens; a 16k target plus sixteen 4k neighbors exceeds it by about 4.8k tokens, consistent with a partial rather than all-or-nothing miss.

4 Corrected Method

4.1 Real and constructed inputs

Arrival time, context length, and generated length come directly from the Azure traces. The traces contain neither program identifiers nor tool durations. We construct programs of $T \in \{1, 4, 16, 64\}$ turns and sweep mean tool delay $\tau \in \{0, 8, 30\}$ s. $T = 1, \tau = 0$ is the unmodified request-level control. Context accumulates across constructed turns. We use seed 20260720.

4.2 State model and service calibration

Each replica separately tracks active KV allocation and reclaimable cached content. A completed turn releases its active allocation but leaves its prefix as LRU content. New active blocks reclaim the oldest parked prefixes at token granularity. Scale-in removes only idle replicas and discards any cached content on them. On return, the program prefills only the missing part of its prefix.

Defaults use the measured 75,216-token capacity, $k_{1/2} = 4.606$, and 38.155 s process cold start. Jointly fitting V1 and V3 gives a service rate of 19,607 token-equivalents/s and decode weight 94.59 relative to prefill. Maximum batch is 32. E7 explicitly sweeps capacity, batch, and policy target.

We simulate 2,400 s of arrivals, exclude a 300 s warm-up from scoring, then stop arrivals and drain for the larger of 600 s or a workload-derived bound. This drain is essential: scoring only to the last arrival mechanically labels late long programs as failures. Every admitted turn completes in the final runs, so goodput is one and does not separate policies.

The SLO is per-turn slowdown at most three. For each workload we search for the smallest static cluster achieving at least 95% SLO attainment. Dynamic cost is reported as GPU-seconds relative to that reference. Unfinished admitted work, if any, is counted as an SLO failure.

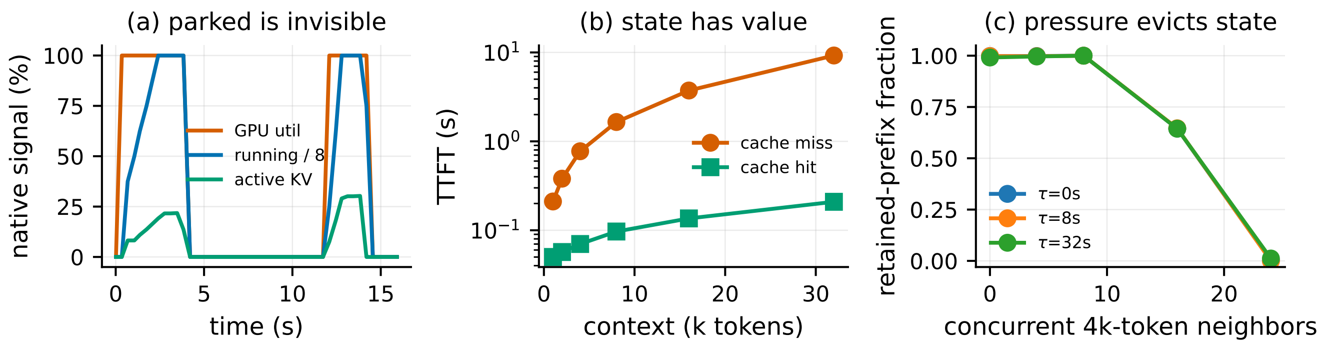

Figure 2: Hardware validation. (a) During tool-call intervals, all three native signals drop to zero. (b) Cache loss has a context-proportional TTFT cost. (c) A parked prefix survives elapsed time but not sufficient concurrent pressure; the score estimates the retained fraction from hit/miss TTFT.

4.3 Policies

We implement HPA on GPU utilisation (70% target and 300 s scale-down stabilisation), KEDA on waiting requests, KV-util on active KV (80% target), a Holt arrival forecaster, and tabular Q-learning with and without parked count in its state [8, 9]. The candidate PARKAWARE policy uses

$$N = \max\left(\left\lceil \frac{\text{running} + \text{queued} + \text{parked}}{B\theta} \right\rceil, \left\lceil \frac{\text{active KV}}{K\theta} \right\rceil\right). \tag{3}$$

It is retained unchanged as the direct test of the original idea. Under the corrected mechanism, its parked term is a conservative seat reservation, not a measurement of pinned memory.

5 Results

5.1 Parked work is common and natively invisible

On a fixed eight-replica cluster, the simulated parked fraction is 60.2% at $(T, \tau) = (4, 4)$, 89.5% at $(8, 8)$, and 92.9% at $(16, 8)$. At those points, respectively 16.2%, 26.2%, and 24.3% of post-warm-up samples contain parked programs while running, GPU, and active-KV signals are all zero. Figure 3 distinguishes how much work is parked from how often its invisibility is complete.

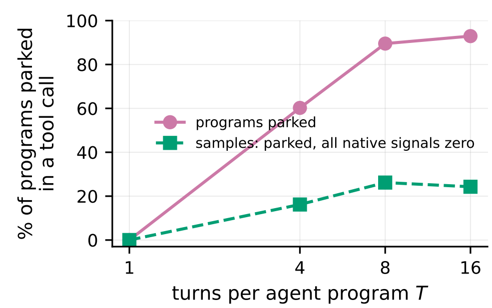

Figure 3: Simulated parked fraction and completely invisible parked intervals on a fixed cluster. Reclaimable cache is tracked internally but is not a native autoscaling signal.

Table 1: Mean over the six coding-trace agentic points ($T > 1, \tau > 0$).

Policy	SLO attainment	GPU / static
HPA (GPU)	0.929	1.836
KEDA (queue)	0.623	1.002
KV-util	**0.941**	1.321
Predictive	0.715	0.909
RL (Q-learning)	0.680	1.019
RL + parked	0.619	0.894
PARKAWARE candidate	0.901	1.417

5.2 Counting parked programs is not a robust policy

Table 1 reverses the uploaded paper's headline. PARKAWARE beats HPA on three of six agentic points and KV-util on two, but loses in the mean. It beats KEDA on five of six, yet usually spends more GPU time. No dynamic policy simultaneously matches the cheapest static SLO configuration and costs less on average. The appropriate conclusion is not that native metrics are sufficient: rather, a parked count by itself is also insufficient.

The result varies by regime. At $(T, \tau) = (4, 8)$, HPA attains 1.000 and PARKAWARE 0.695. At $(16, 30)$, PARKAWARE attains 1.000 and HPA 0.994. At $(64, 8)$, KV-util and PARKAWARE both attain 1.000 while HPA is 0.792. Such crossovers are exactly what a single occupancy proxy should produce when prefix value depends on context size, return timing, cache pressure, and service batching.

5.3 Pressure dominates scale-in damage

At $T = 8, \tau = 8$, HPA recomputes 2.19M tokens, of which only 0.324M are attributed to scale-in. KEDA, KV-util, and

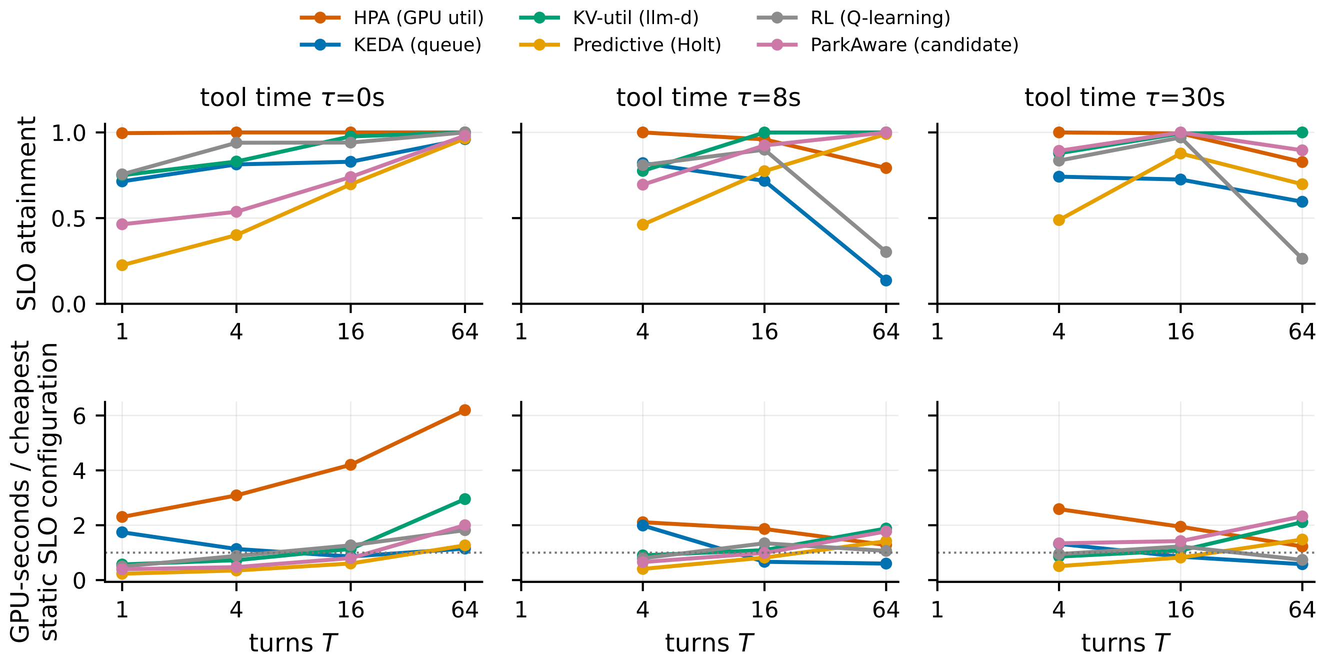

Figure 4: SLO attainment and GPU cost relative to the cheapest static SLO-meeting cluster. The corrected model yields no universal dynamic-policy winner; ParkAware is a candidate baseline, not "ours" with a claimed win.

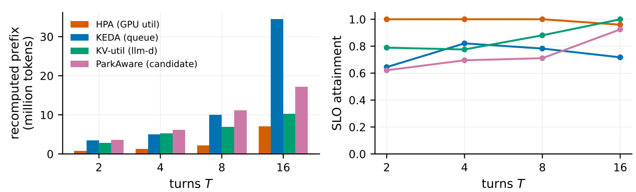

Figure 5: Recomputed prefix tokens and SLO on the coding trace. Recompute includes partial pressure eviction and cache loss on scale-in.

PARKAWARE recompute 9.96M, 6.88M, and 11.17M tokens respectively; their scale-in components are only 0.514M, 0.176M, and 0.361M. Thus routine cache pressure is the dominant source of lost prefix work in this model. The old equation that charged every parked context only when a replica was removed described the wrong mechanism.

5.4 Cold start and workload change the ordering

At the measured 38.155 s process cold start, HPA, KEDA, Predictive, and PARKAWARE attain 1.000, 0.782, 0.521, and 0.711 at $(T, \tau) = (8, 8)$. At 180 s they attain 0.605, 0.674, 0.399, and 0.644. The original monotone ParkAware advantage does not survive correction: KEDA is best among these four at 180 s, and HPA is best at the measured point.

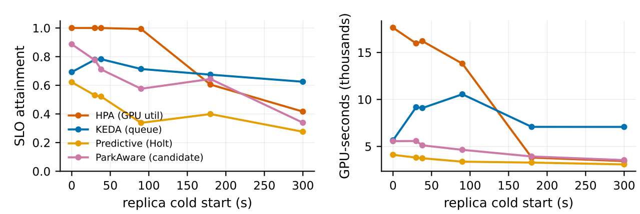

Figure 6: Cold-start sensitivity at $T = 8, \tau = 8$. The measured 14B point is 38.155 s; larger values are sensitivity cases, not measurements.

On the conversation trace constructed as $T = 8, \tau = 8$, HPA attains 1.000, KV-util 0.921, and PARKAWARE 0.974; KEDA falls to 0.280. On the coding trace at the same point, the values are 1.000, 0.881, and 0.711 for the first, third, and fourth policies. A program-level policy cannot be evaluated independently of the token and arrival distribution beneath the constructed program structure.

5.5 Sensitivity confirms that no scalar signal is enough

At measured capacity and batch 32, HPA/KV-util/PARKAWARE attain 1.000/0.881/0.711. At batch 16 the same values are 1.000/0.930/0.942; at batch 64, 0.998/0.796/0.596. Sweeping capacity from 31k to 500k also changes the ordering, and changing ParkAware's target from 0.5 to 0.9 moves its attainment from 0.845 to 0.633. The candidate is sensitive to the resource regime and control target, contrary to the original robustness claim.

6 Implications

Expose cache state, not only allocation. Active-KV utilisation correctly reports allocated blocks, but an autoscaler also needs the amount, age, ownership, and eviction priority of reclaimable prefix content. Conflating the two would make admission control unsafe; exporting both would preserve the distinction.

Value is probabilistic. A parked prefix is worth approximately its miss penalty times the probability that the program returns before pressure evicts it. Tool type, transcript size, recent reuse, and cache competition may predict that value. A raw parked count assigns the same value to a 1k prefix and a 32k prefix despite a measured 56-fold difference in recompute penalty.

Scale-in should be cache-aware, not cache-fearing. Scale-in is destructive when it targets a replica with valuable surviving prefixes, but keeping a replica for already-evicted or unlikely-to-return state wastes capacity. A practical controller should coordinate replica draining with prefix placement and eviction metadata rather than prohibit scale-in based on program count.

7 Threats to Validity

Single hardware configuration. Four modelling assumptions are measured on a single vLLM instance. One assumption was falsified by that measurement and the model corrected accordingly. Results may differ across engines, versions, models, cache managers, and hardware. V4 measures a process restart with warm host page cache, not a machine reboot or disk-cold start.

Constructed program structure. Azure supplies request records, not agent identities or tool intervals. Our T and τ are constructed. Sweeps and a $T = 1$ control expose, but do not eliminate, this limitation.

Simulator rather than cluster replay. Hardware measurements calibrate the mechanism and service curve; E2–E7 are still simulations. The model uses LRU pressure, one model per replica, and simplified placement. Production schedulers may migrate, preempt, offload, or explicitly pin cache state.

Policy implementations. The policies follow their documented signal and standard control forms, but deployment-specific tuning can change results. The tabular RL baseline is deliberately conventional, not a claim about all learning-based controllers. We present ParkAware as a falsified candidate rather than a production recommendation.

8 Related Work

PagedAttention and vLLM make KV management efficient within a serving instance [3]. Agent-serving systems increasingly schedule whole programs rather than isolated requests; Autellix, for example, uses cumulative program service time [4]. This work addresses the interface between that program state and cluster-level elasticity.

LLM autoscaling work optimizes token-aware or disaggregated serving. TokenScale scales on token velocity, while ServerlessLLM attacks model-loading cold start directly [5, 6]. DistServe and DynamoLLM optimize cluster organization and goodput [11, 12]. Our measurement shows that active request metrics omit a distinct object: reclaimable, application-owned prefix value between requests.

Classical elasticity separates reactive, proactive, and learning-based controllers [7, 9]. RL resource allocation has a long history [8]. Our corrected result is compatible with that literature's main lesson: policy quality depends on the state representation. Here, neither native active-resource signals nor a parked-program count represents prefix survival and value.

9 Conclusion

A parked agent does not hard-hold KV in native vLLM, but it may retain a cheap path back into execution. That path has measurable value, is invisible to native autoscaling metrics, and disappears under cache pressure. Correcting this distinction invalidates both our original mechanism and its claimed ParkAware advantage. The robust conclusion is narrower and more useful: agentic autoscaling needs an explicit signal for reclaimable prefix state and its reuse probability. We release the failed assumption and corrected negative result so subsequent designs can start from the measured system rather than a convenient fiction.

A Sensitivity Figure

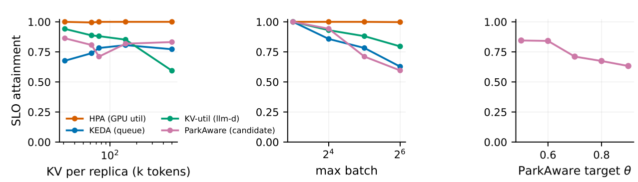

Figure 7: Sensitivity at $T = 8, \tau = 8$: KV capacity, maximum batch, and the candidate ParkAware target.

References

[1] P. Patel et al. Splitwise: Efficient generative LLM inference using phase splitting. In *ISCA*, 2024. Traces: https://github.com/Azure/AzurePublicDataset.

[2] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

[3] W. Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

[4] M. Luo et al. Autellix: An efficient serving engine for LLM agents as general programs. arXiv:2502.13965, 2025.

[5] R. Lai et al. TokenScale: Timely and accurate autoscaling for disaggregated LLM serving with token velocity. arXiv:2512.03416, 2025.

[6] Y. Fu et al. ServerlessLLM: Low-latency serverless inference for large language models. In *OSDI*, 2024.

[7] N. Herbst, S. Kounev, R. Reussner. Elasticity in cloud computing: What it is, and what it is not. In *ICAC*, 2013.

[8] E. Barrett, E. Howley, J. Duggan. Applying reinforcement learning towards automating resource allocation and application scalability in the cloud. *Concurrency and Computation*, 25(12), 2013.

[9] Y. Garí et al. Reinforcement learning-based application autoscaling in the cloud: A survey. *Engineering Applications of Artificial Intelligence*, 2021.

[10] Polar: Agentic RL on any harness at scale. arXiv:2605.24220, 2026.

[11] J. Stojkovic et al. DynamoLLM: Designing LLM inference clusters for performance and energy efficiency. In *HPCA*, 2025.

[12] Y. Zhong et al. DistServe: Disaggregating prefill and decoding for goodput-optimized LLM serving. In *OSDI*, 2024.